

# **DesiNuts (Online Web Based Application)**

**By:**

**Zaib Noorkhan Shaikh –**

**AF05264683**

**Siddiqui Mohd Hasnain**

**Abdul Samad –**

**AF05250529**

# Index

| Sr.no | Topic           | Pageno |
|-------|-----------------|--------|
| 1     | TitleofProject  | 1      |
| 2     | Acknowledgement | 3      |
| 3     | Abstract        | 4      |
| 4     | Introduction    | 5      |
| 5     | Objective       | 6      |
| 6     | SystemAnalysis  | 7      |
| 7     | SystemDesign    | 23     |
| 8     | Frontend design | 25     |
| 9     | Backend Design  | 32     |
| 10    | Databases       | 39     |
| 11    | FutureScope     | 42     |
| 12    | Bibliography    | 43     |

# Acknowledgement

The project“**DesiNuts**”is the Project work carried out by

| Name                                     | EnrollmentNo      |
|------------------------------------------|-------------------|
| <b>Zaib Noorkhan Shaikh</b>              | <b>AF05264683</b> |
| <b>Siddiqui Mohd Hasnain Abdul Samad</b> | <b>AF05250529</b> |

## UNDER THE GUIDANCE

We are thankful to my project guide for guiding me to complete the project. His suggestions and valuable information regarding the formation of the project report have provided me a lot of help in completing the project and its related topics.

We are also thankful to my family members and friends who were always there to provide support and moral boost-up.

# Abstract

The project called **DesiNuts** is about creating an e-commerce website for selling premium dry fruits and nuts online. This system provides a digital platform where customers can conveniently explore products, view detailed product information, and purchase their preferred dry fruits and nuts. It replaces the traditional way of purchasing products by providing an organized online shopping experience with a professional and user-friendly interface.

The website allows users to browse different products, explore the shop section, manage products through the shopping cart, and proceed through the ordering workflow. It also includes a dedicated **Bulk Orders** section for customers interested in purchasing products in larger quantities. The system is developed using **React.js** for the frontend, **Node.js and Express.js** for the backend, and **MongoDB** for storing product, user, and order-related data.

The system is designed to provide a simple, responsive, and convenient shopping experience. It helps organize product information, streamline the shopping process, and provide customers with easy access to dry fruits and nuts through an online platform. The project focuses on accessibility, efficient product management, and improving the overall digital shopping experience.

# Introduction

In today's age of Information Communication and Technology, we are surrounded by technology in almost every aspect of our daily lives. From communication and education to shopping and business, technology plays an important role in making activities faster and more convenient. One of the most important developments in this technological revolution is the growth of web applications. These applications allow people to access services and information from anywhere, anytime, through the internet.

Our project aims to develop an e-commerce web application named **DesiNuts**, which provides customers with a convenient platform for purchasing premium dry fruits and nuts online. The system allows users to explore different products, view product details, manage their shopping cart, and proceed through the ordering process. It also provides a dedicated Bulk Orders section for customers interested in purchasing products in larger quantities. The application is designed to provide an organized, responsive, and user-friendly shopping experience.

The **DesiNuts** project uses modern web technologies to provide an efficient digital shopping platform. The frontend is developed using **React.js**, while **Node.js** and **Express.js** are used for backend development and handling application services. **MongoDB** is used as the database for storing product, user, and order information. This combination of technologies helps create a dynamic and scalable e-commerce application while providing a smooth experience for customers.

# Objective of the Present Work

The objectives of this project are as follows:

- To develop a web-based e-commerce application for selling premium dry fruits and nuts online.
- To provide customers with an easy and convenient online shopping experience.
- To display a well-organized product catalog with detailed information about available products.
- To provide an interactive and visually appealing interface for browsing and purchasing products.
- To allow customers to add products to the shopping cart and manage selected items easily.
- To provide a dedicated Bulk Orders section for customers interested in purchasing products in larger quantities.
- To manage product, user, and order-related information efficiently using a database.
- To develop a responsive and user-friendly platform that can be accessed through different devices.
- To streamline the online shopping workflow and improve the overall customer experience.

# System Analysis

## **Problem Definition**

Many specialty food and dry fruit retailers struggle with managing extensive product catalogs, processing bulk orders efficiently, and directly capturing customer sentiment regarding product quality. Traditional local or manual sales methods limit market reach, lack structured digital inventory control, and make it difficult to track customer feedback trends at scale.

This project solves these problems by providing a centralized, full-stack MERN (MongoDB, Express.js, React.js, Node.js) web application designed specifically for retailing premium dry fruits and nuts online. The system makes inventory management seamless for administrators while giving users a clean, responsive, and intuitive platform to browse premium goods, manage their shopping carts, and place retail or bulk orders effortlessly.

## **Preliminary Investigation**

### ***Purpose***

The **DesiNuts** web portal is designed to digitize the retail and bulk distribution of premium dry fruits and nuts through an accessible online platform. It replaces traditional manual ordering methods by providing administrators with structured inventory management tools while offering users a modern, smooth, and premium digital shopping experience.

### ***Benefits***

- **Instant Product Access:** Users can browse a wide variety of premium dry fruits, nuts, and gift hampers organized by categories.

- **Efficient Content & Inventory Management:** Admins can easily add, update, and manage product listings and stock details without delays.
- **Flexible Ordering:** Supports both individual retail shopping carts and a dedicated bulk orders section for large-scale buyers.
- **Streamlined User Experience:** Clean UI inspired by industry standards, ensuring fast loading speeds and frictionless checkout pathways.

### **Proposed System**

The proposed system offers a robust, scalable, and modern full-stack architecture:

- **Users** can browse products, manage their shopping carts, submit bulk order requests, and receive a smooth, responsive UI experience across devices.
- **Administrators** can control the product catalog, monitor incoming orders, manage customer inquiries, and oversee inventory updates.

- **The Architecture** utilizes React.js for a dynamic frontend coupled with Node.js and Express.js on the backend, backed by MongoDB for secure, flexible data storage.

## **Feasibility Study**

### ***1. Technical Feasibility***

- Built using **React.js** for a responsive frontend and **Node.js/Express.js** for a scalable RESTful backend.
- **MongoDB** handles product catalogs, user sessions, and order information with high flexibility and scalability.
- Hosted on reliable cloud infrastructure to support consistent traffic and fast response times.

### ***2. Economic Feasibility***

- Utilizes **open-source technologies** (Node.js, Express, React, MongoDB libraries), significantly minimizing licensing and software costs.
- Streamlines administrative workflows, reducing manual order-processing overhead and human error.

### ***3. Operational Feasibility***

- Designed with a **minimalist, user-friendly interface** that requires zero technical training for customers to navigate and purchase products.
- Administrative panels are structured intuitively for easy inventory and order tracking.

### ***4. Schedule Feasibility***

- The development roadmap is broken down into structured phases (design, frontend integration, backend logic, database linking, and testing) to meet target deadlines comfortably.
- Modular MERN structure allows components (such as the cart, product catalog, and bulk order form) to be built and tested incrementally.

### ***5. Social Feasibility***

- Promotes healthy, quality-conscious dietary habits by making premium natural dry fruits easily accessible to everyone.
- Offers a safe, transparent e-commerce space with clear pricing, product descriptions, and straightforward ordering processes.

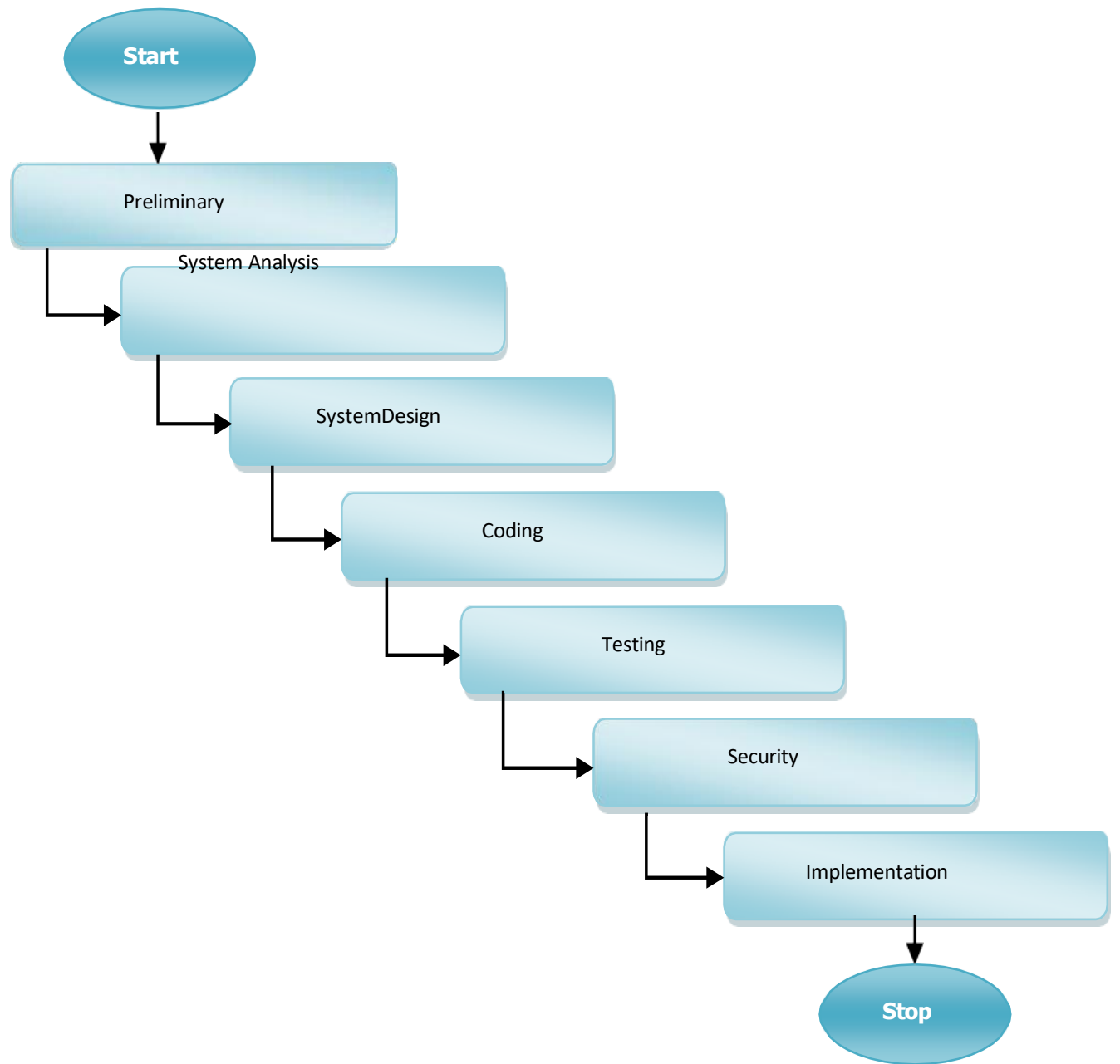
## **Project Planning**

### ***Purpose of Project Planning***

Project planning ensures that the creation of the **DesiNuts** e-commerce portal follows a disciplined, organized workflow. It defines task allocations, milestones, timelines, and architectural boundaries to deliver a stable application efficiently.

### ***Phases Covered in the Plan***

1. **Preliminary Investigation:** Defining project scope, target audience, and business requirements (inspired by benchmarks like *Dry Fruit House*).
2. **System Analysis:** Identifying challenges in traditional retail, defining database schemas, and mapping out user flows for retail vs. bulk orders.
3. **System Design:** Structuring MERN modules, designing wireframes with clean color palettes, and organizing component architecture.
4. **Coding:** Developing the frontend components using React and implementing backend REST APIs with Node.js and Express.
5. **Security:** Implementing secure user authentication, input validation, and protected admin routes.
6. **Testing:** Performing unit tests on API endpoints, frontend integration testing, and cross-browser responsiveness checks.
7. **Implementation:** Deploying the full-stack application to production servers and ensuring seamless live operation.



## Software Requirement Specification (SRS): DesiNuts

The Software Requirement Specification (SRS) outlines the fundamental requirements of the **DesiNuts** E-Commerce Portal to ensure efficient functionality, a seamless user experience, and easy maintainability.

### System Overview

The DesiNuts portal allows users to browse, select, and purchase premium dry fruits and nuts online, supporting both individual retail shopping and large-scale bulk orders. The system is structured into three core modules:

1. **User Module:** Enables customers to browse products, search for specific dry fruits or gift hampers, manage their shopping carts, and place retail or bulk orders.
2. **Admin Module:** Provides comprehensive tools for product management, inventory tracking, order processing, and customer inquiry management.
3. **Inventory & Order Management Module:** Tracks stock levels in real-time and manages the status of incoming retail shipments and bulk order requests.

## Software & Hardware Requirements

### Software Requirements

- **Frontend:** React.js, Tailwind CSS / Modern CSS for a responsive, clean, and minimalist UI.
- **Backend:** Node.js with Express.js for handling RESTful APIs, routing, and business logic.
- **Database:** MongoDB for storing product catalogs, user accounts, shopping carts, and order histories.
- **Authentication:** JSON Web Tokens (JWT) and bcrypt for secure user login and password hashing.
- **WebServer/Hosting:** Node.js runtime environment hosted on a cloud platform (e.g., Vercel, Render, or AWS).

## ***Hardware Requirements***

- **Processor:** Intel i5 or higher (or equivalent multi-core processor).
- **RAM:** Minimum 8 GB.
- **Storage:** At least 100 GB for database storage, product media assets, and server logs.
- **Connectivity:** High-speed internet access for real-time inventory synchronization and payment/order processing.

## **Functional Requirements**

### ***1. User Module***

Users can:

- Browse premium dry fruits, nuts, seeds, and gift hampers categorized by type, price, and health benefits.
- Search for specific products instantly using keyword search bars and filters.
- Manage a digital shopping cart (add items, update quantities, remove items).
- Place retail orders or submit dedicated **bulk order requests** for large-scale purchases.
- View order history and track the status of their current purchases.

### ***2. Admin Module***

Admins have full control over the platform and can:

- **Secure Login System:** Access the admin dashboard securely through token-based authentication.
- **Product Catalog Management:** Add, edit, delete, and update product details, pricing, categories, and stock availability.
- **Order Management:** View, accept, process, and update the fulfillment status of customer orders (Retail & Bulk).
- **Inventory Monitoring:** Track stock levels to prevent out-of-stock listings and manage reorder alerts.
- **Customer Inquiry Control:** Review and respond to bulk order inquiries submitted through the portal.

### ***3. Order & Inventory Management Module***

This system component ensures that:

- Stock counts decrease automatically upon successful order placement.
- Admins receive real-time notifications for new bulk order requests.
- Order statuses transition smoothly from *Pending* to *Processing*, *Shipped*, and *Delivered*.

### **Software Engineering Paradigm**

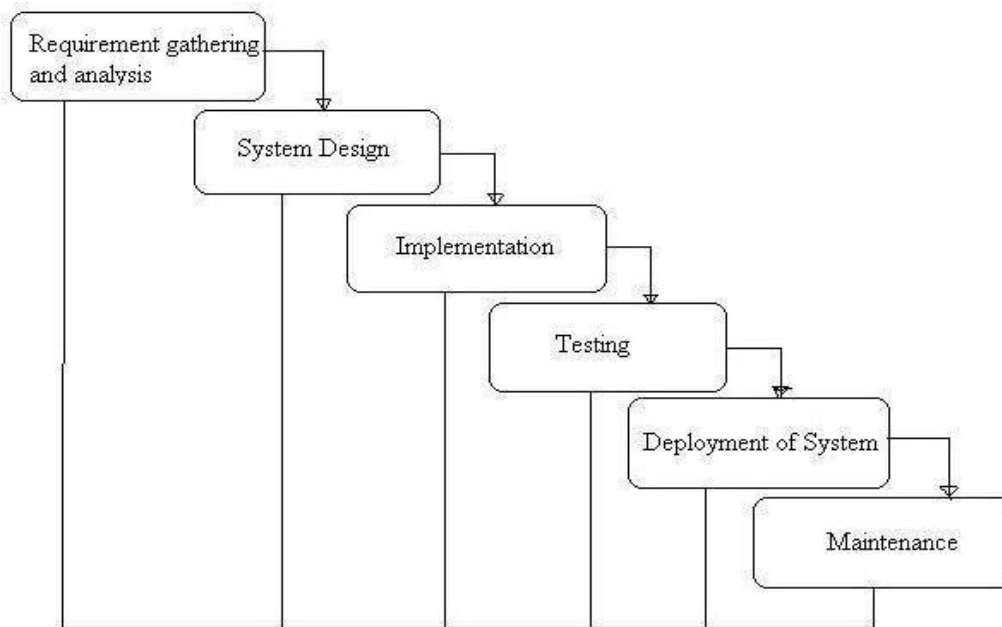
The development of the **DesiNuts** E-Commerce Portal follows a structured approach to ensure efficiency, reliability, and code maintainability. The chosen paradigm helps streamline the project by defining clear milestones while allowing iterative improvements during full-stack integration.

Development Model: Adapted Waterfall Model

While the traditional Waterfall model is linear, this project incorporates an **iterative feedback mechanism** between the React frontend development and Node.js backend integration to refine features seamlessly.

### ***Key Adaptations in the Waterfall Model:***

1. **Structured Phase Progression:** Each stage (wireframing, schema design, frontend coding, backend API creation) follows a defined sequence.
2. **Iterative Refinements:** Feedback loops allow adjustments during component integration—such as tweaking cart state management or database query performance.
3. **Defined Milestones:** Each stage reaches completion before moving to the next phase (e.g., database schema finalization precedes API endpoint development).
4. **Flexible Adjustments:** Modular MERN architecture permits overlapping tasks when necessary to enhance development speed and code reuse.



## Phases of Development

1. Requirement Analysis & System Study
  - Identifying project goals, challenges, and functional specifications.
  - Gathering stakeholder requirements and defining core functionalities.
2. System Design
  - Structuring the **database, modules, and architecture**.
  - Designing **user interfaces** for optimal accessibility.
3. Implementation (Coding)
  - Backend development using **Python (Django)**.
  - Frontend design using **HTML, CSS, Bootstrap**.
  - Database integration with **MySQL**.
  - Sentiment analysis integration with **TextBlob**.
4. Testing & Debugging
  - Unit testing, integration testing, and usability checks.
  - Debugging for performance improvements.
5. Deployment & Maintenance
  - Hosting on a scalable environment.
  - Continuous updates for feature enhancements.

### **DataFlow Diagram:**

A Data Flow Diagram (DFD) is a traditional visual representation of the information

flows within a system. A neat and clear DFD can depict the right amount of the system requirement graphically. It can be manual, automated, or a combination of both.

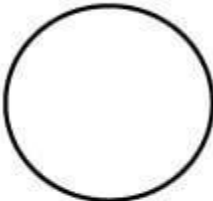
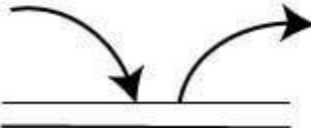
It shows how data enters and leaves the system, what changes the information, and where data is stored.

The objective of a DFD is to show the scope and boundaries of a system as a whole. It may be used as a communication tool between a system analyst and any person who plays a part in the order that acts as a starting point for redesigning a system. The DFD is also called as a data flow graph or bubble chart.

### **The following observations about DFDs are essential:**

1. All names should be unique. This makes it easy to refer to elements in the DFD.
2. Remember that DFD is not a flow chart. Arrows in a flow chart represent the order of events; arrows in DFD represent flowing data. A DFD does not involve any order of events.
3. Suppress logical decisions. If we ever have the urge to draw a diamond-shaped box in a DFD, suppress that urge! A diamond-shaped box is used in flow charts to represent decision points with multiple existing paths of which the only one is taken. This implies an ordering of events, which makes no sense in a DFD.
4. Do not become bogged down with details. Defer error conditions and error handling until the end of the analysis.

Standard symbols for DFDs are derived from the electric circuit diagram analysis and are shown in fig:

| Symbol                                                                             | Name                                | Function                                                                                                       |
|------------------------------------------------------------------------------------|-------------------------------------|----------------------------------------------------------------------------------------------------------------|
|   | Data flow                           | Used to Connect Processes to each other, to sources or Sinks; the arrow head indicates direction of data flow. |
|   | Process                             | Performs Some transformation of Input data to yield output data.                                               |
|   | Source of Sink<br>(External Entity) | A Source of System inputs or Sink of System outputs.                                                           |
|  | Data Store                          | A repository of data; the arrow heads indicate net inputs and net outputs to store.                            |

### Symbols for Data Flow Diagrams

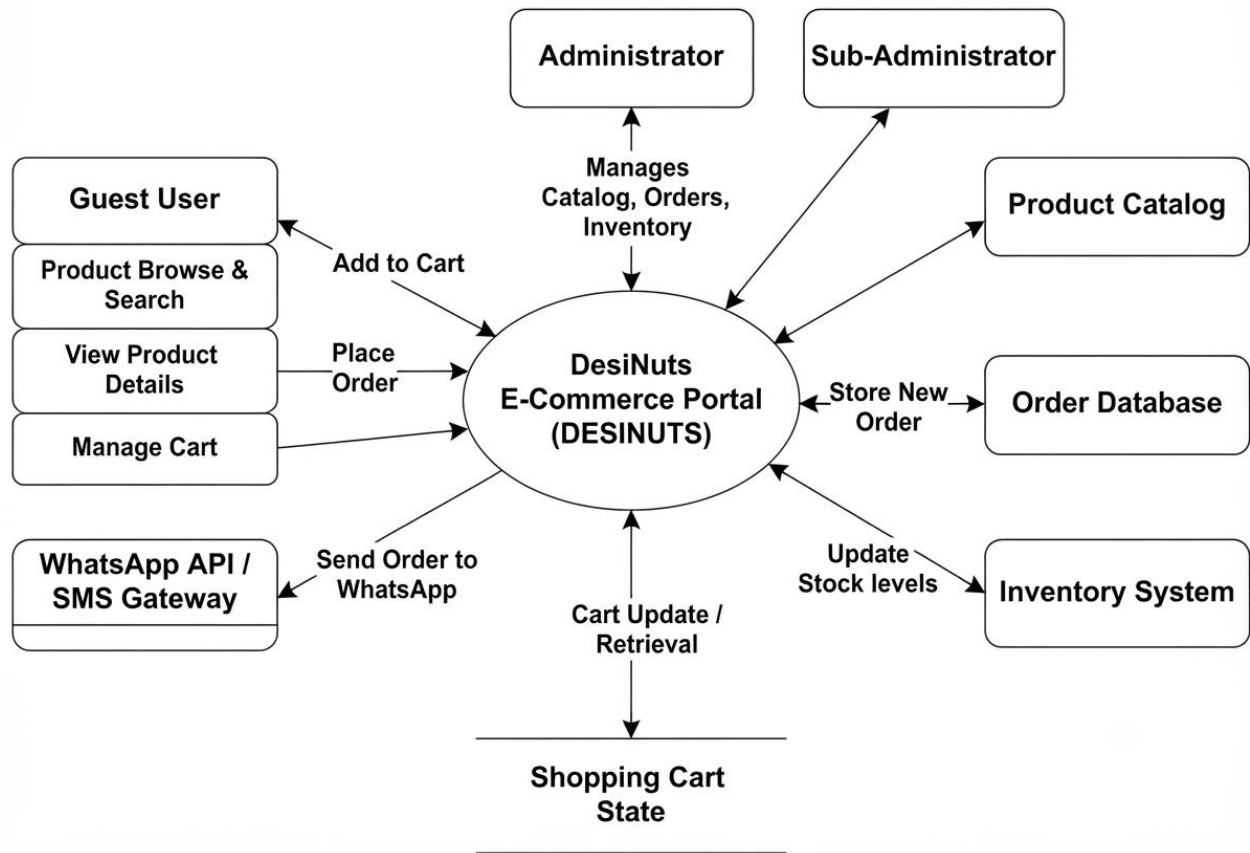
**Circle:** A circle (bubble) shows a process that transforms data inputs into data outputs.

**Data Flow:** A curved line shows the flow of data into or out of a process or data store.

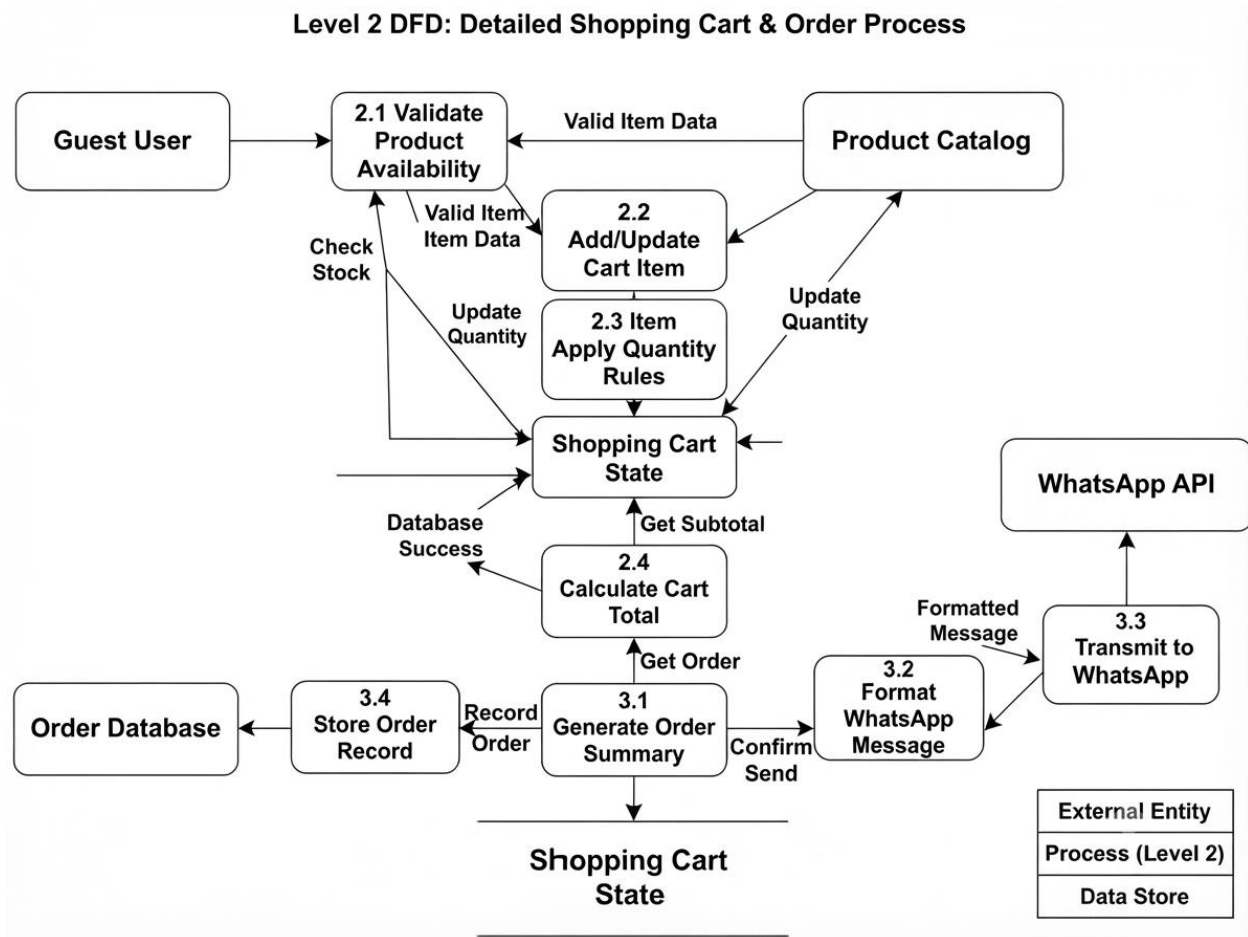
**Data Store:** A set of parallel lines shows a place for the collection of data items. A data store indicates that the data is stored which can be used at a later stage or by the other processes in a different order. The data store can have an element or group of elements.

**Source or Sink:** Source or Sink is an external entity and acts as a source of system inputs or sink of system outputs.

### Zero Level DFD: DesiNuts E-Commerce Portal (Adapted)



The Zero Level DFD of the **DesiNuts E-Commerce Portal (DESINUTS)** illustrates the high-level processes and external entities that interact with the system. It highlights the flow of data between the central portal and its core functional modules, including **Product Catalog**, **Shopping Cart State**, and **Order Management**.

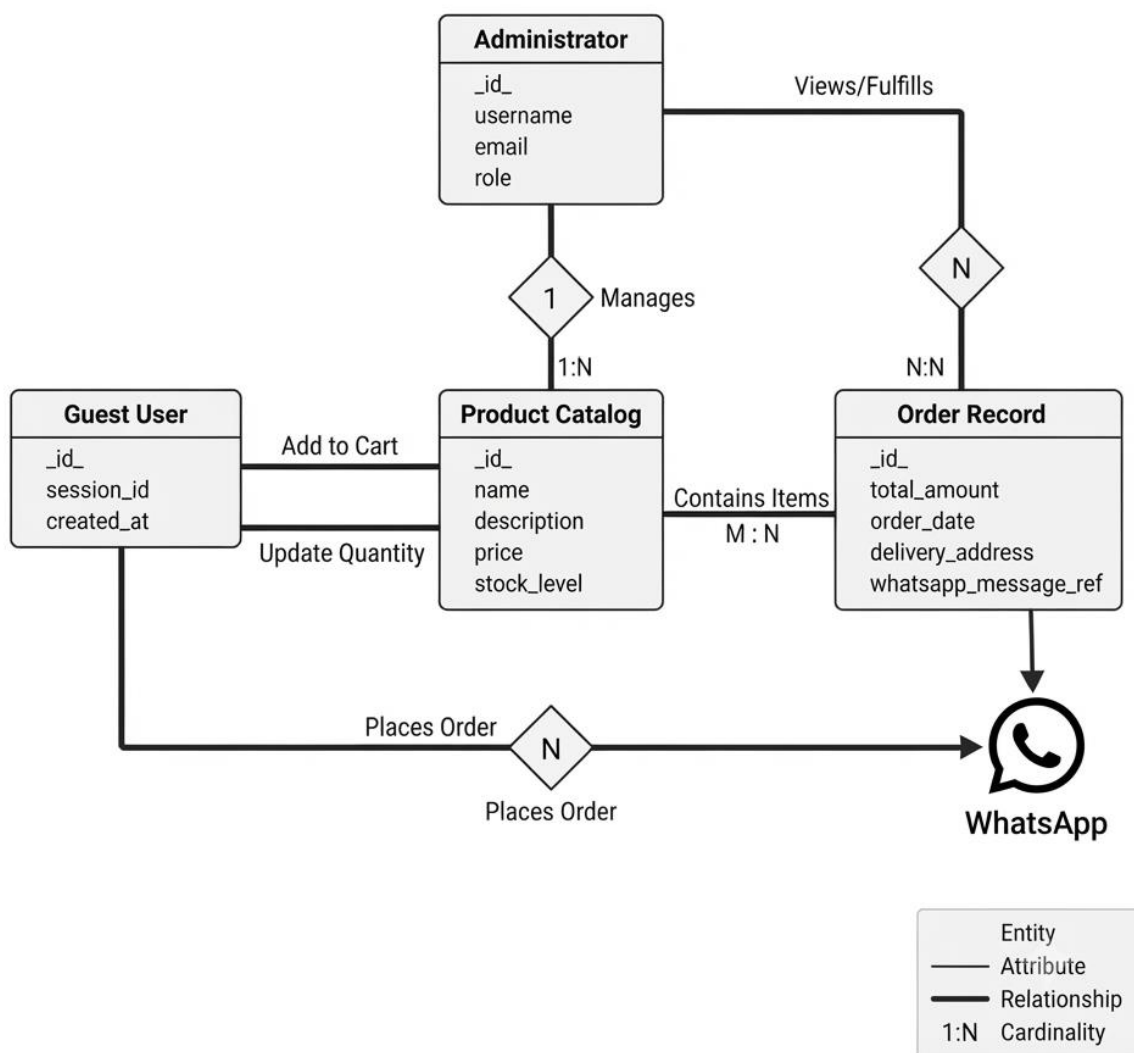


This Second-Level DFD represents the internal workflow for shopping cart management and order processing in the **DesiNuts portal**. After a guest user views product details and checks stock availability, the system validates item data from the product catalog and allows them to add or update items in the cart. Additionally, the system calculates cart totals, generates an order summary, formats a message for the WhatsApp API, and simultaneously records the order in the database.

## ERdiagram

TheERDiagramoftheOnlineNewsPortalSystemshowskeyrelationshipsbetween **Users**, **Admins**, **News Articles**, **Categories**, **Comments**, and **Sentiment Analysis**, ensuring structured data management and insightful moderation.

## Simplified ERD: DesiNuts E-Commerce Portal



# System Design

The system design for the **DesiNuts** e-commerce portal outlines the structural architecture, component mapping, and data flow required to deliver a seamless shopping and bulk-ordering experience without traditional user login barriers.

## System Architecture Overview

The application follows a modern **MERN (MongoDB, Express.js, React.js, Node.js)** architecture:

- **Frontend (Client-Side):** Developed using React.js and Tailwind CSS/modern styling to provide a responsive, minimalist, and rapid user interface across mobile and desktop devices.
- **Backend (Server-Side):** Powered by Node.js and Express.js, handling routing, cart management middleware, product APIs, and communication bridges.
- **Database Layer:** Uses MongoDB for flexible and scalable document storage of product catalogs, inventory lists, and customer orders.
- **External Integration Layer:** Interfaces with the **WhatsApp API / SMS Gateway** to enable direct, one-click order placement and customer inquiries.

## Core System Modules (Adapted for DesiNuts)

### 1. User & Guest Shopping Module

- **Product Catalog & Browsing:** Users can view premium dry fruits and nuts categorized by type, price, and gift sets.
- **Cart State Management:** Client-side state handling allows guest users to add items, modify quantities, and review subtotal amounts instantly without forcing a registration wall.
- **WhatsApp Direct Ordering:** Instead of complex payment gateways or mandatory user login profiles, final cart items are dynamically formatted into an automated text template and sent directly to the business via the WhatsApp API.

### 2. Administrator Module

Admins maintain full control over store operations via a secured dashboard:

- **Authentication:** Secure JSON Web Token (JWT) based login for administrative access.
- **Product & Inventory Control:** Full CRUD operations (Add, Edit, Delete) for managing dry fruit inventories, pricing, and stock levels.
- **Order & Inquiry Management:** Tracks incoming order records and bulk purchase requests generated through the portal.

### *3. Order & Inventory Processing Module*

- Automatically links frontend cart actions with backend data stores.
- Ensures stock counts remain synchronized and records transaction histories securely in the database for fulfillment tracking.

# Frontend Design

## Resgiter Page:

[Home](#)[Shop](#)[Bulk Orders](#)[About Us](#)[Contact Us](#)[Login](#)

### Create Account

Join Desi Nuts for a better shopping experience

Already have an account? [Login here](#)

## Login Page:

### Welcome Back

Login to your Desi Nuts account

Don't have an account? [Register here](#)

## Home Page:

[Home](#)[Shop](#)[Bulk Orders](#)[About Us](#)[Contact Us](#)[zaib](#)

### Our Categories



## Shop Page:

[Home](#)

**Shop**

## Bulk Orders

## About Us

## Contact Us

 zaib



**Cashew Nuts (Kaju) Medium  
Sized**

**₹130** / 100 gram

100 gram 

**Add to cart >**



### Premium Jumbo Almonds

**₹110** / 100 gram

100 gram 

**Add to cart >**



### Premium Pistachios

**₹125** / 100 gram

100 gram 

**Add to cart >**



### Pistachio Kernels (Peeled)

**₹145** / 100 gram

100 gram ▼

**Add to cart >**

## BulkOrder Page:

[Home](#)[Shop](#)[Bulk Orders](#)[About Us](#)[Contact Us](#)[zaib](#)

Let us get the  
business started!

Reach out us today.

Follow us



Name\*

Write your bulk order requirements here (e.g. quantity, items,  
delivery location)...

[Submit Bulk Inquiry >](#)

## Contact Us Page:

[Home](#)[Shop](#)[Bulk Orders](#)[About Us](#)[Contact Us](#)[zaib](#)

### Contact Details

  
Desi Nuts Agro Farms, Survey No. 142, MIDC Agricultural Zone,  
NH-48 Bypass,  
near Khedshivapur, Bhiwandi, Maharashtra - 421302

  
[info@Desinuts.com](mailto:info@Desinuts.com)

  
[+91-7483600232](tel:+91-7483600232)

# Add to Cart Page:



HomeShopBulk OrdersAbout UsContact Us

 zaib

### Your Shopping Cart



Cashew Nuts (Kaju) Medium Sized

100 gram ₹130 each

-

1

+

₹130





Premium Jumbo Almonds

100 gram ₹110 each

-

1

+

₹110



Subtotal

Estimated Shipping

Grand Total

₹240

₹60

₹300

[← Continue Shopping](#)

[Proceed to Checkout](#)

# Backend Design

## Authentication Control:

```
using DesiNuts.Api.Data;
```

```
using DesiNuts.Api.Models;
```

```
using Microsoft.AspNetCore.Mvc;
```

```
namespace DesiNuts.Api.Controllers
```

```
{
```

```
    [ApiController]
```

```
    [Route("api/[controller]")]
```

```
    public class AuthController : ControllerBase
```

```
    {
```

```
        private readonly ApplicationDbContext _context;
```

```
        public AuthController(ApplicationDbContext context)
```

```
        {
```

```
            _context = context;
```

```
        }
```

```
        [HttpPost("register")]
```

```
public IActionResult Register(User newUser)  
{  
    if (string.IsNullOrEmpty(newUser.FullName) ||  
        string.IsNullOrEmpty(newUser.Email) ||  
        string.IsNullOrEmpty(newUser.Password))  
    {  
        return BadRequest(new { message = "Full name, email  
and password are required" });  
    }  
  
    bool emailTaken = _context.Users.Any(u => u.Email ==  
newUser.Email);  
    if (emailTaken)  
    {  
        return BadRequest(new { message = "An account with this  
email already exists" });  
    }  
  
    _context.Users.Add(newUser);  
    _context.SaveChanges();
```

```
        return Ok(new { message = "Account created successfully.  
You can now log in." });
```

```
    }
```

```
[HttpPost("login")]
```

```
public IActionResult Login(User loginUser)
```

```
{
```

```
    var user = _context.Users.FirstOrDefault(u =>  
        u.Email == loginUser.Email && u.Password ==  
loginUser.Password);
```

```
    if (user == null)
```

```
    {
```

```
        return BadRequest(new { message = "Invalid email or  
password" });
```

```
    }
```

```
    HttpContext.Session.SetInt32("UserId", user.Id);
```

```
    HttpContext.Session.SetString("FullName", user.FullName);
```

```
    return Ok(new { message = "Login successful", fullName =  
user.FullName });
```

```
}
```

```
[HttpPost("logout")]
```

```
public IActionResult Logout()
```

```
{
```

```
    HttpContext.Session.Clear();
```

```
    return Ok(new { message = "Logged out" });
```

```
}
```

```
[HttpGet("me")]
```

```
public IActionResult Me()
```

```
{
```

```
    int? userId = HttpContext.Session.GetInt32("UserId");
```

```
    if (userId == null)
```

```
    {
```

```
        return Ok(new { loggedIn = false });
```

```
    }
```

```
    string? fullName =
```

```
    HttpContext.Session.GetString("FullName");
```

```
    return Ok(new { loggedIn = true, fullName });
```

## **Order Control:**

```
using DesiNuts.Api.Data;  
using DesiNuts.Api.Models;  
using Microsoft.AspNetCore.Mvc;  
using Microsoft.EntityFrameworkCore;  
  
namespace DesiNuts.Api.Controllers  
{  
  
    [ApiController]  
    [Route("api/[controller]")]  
    public class OrdersController : ControllerBase  
    {  
        private readonly ApplicationDbContext _context;  
  
        public OrdersController(ApplicationDbContext context)  
        {  
            _context = context;  
        }  
  
        [HttpPost]  
        public IActionResult PlaceOrder(Order order)
```

```
{  
    if (string.IsNullOrEmpty(order.CustomerName) ||  
        string.IsNullOrEmpty(order.Phone) ||  
        string.IsNullOrEmpty(order.Address))  
    {  
        return BadRequest(new { message = "Name, phone and  
address are required" });  
    }  
  
    if (order.Items == null || order.Items.Count == 0)  
    {  
        return BadRequest(new { message = "Cart is empty" });  
    }  
  
    decimal total = 0;  
  
    foreach (var item in order.Items)  
    {  
        var product = _context.Products.Find(item.ProductId);  
        if (product == null)  
        {
```

```
        return BadRequest(new { message = $"Product  
{item.ProductId} not found" });
```

```
    }
```

```
        item.ProductName = product.Name;
```

```
        item.Price = GetPriceForWeight(product,  
item.WeightGrams);
```

```
        total += item.Price * item.Quantity;
```

```
    }
```

```
    order.TotalAmount = total;
```

```
    _context.Orders.Add(order);
```

```
    _context.SaveChanges();
```

```
    return Ok(order);
```

```
}
```

```
[HttpGet]
```

```
public IActionResult GetOrders()
```

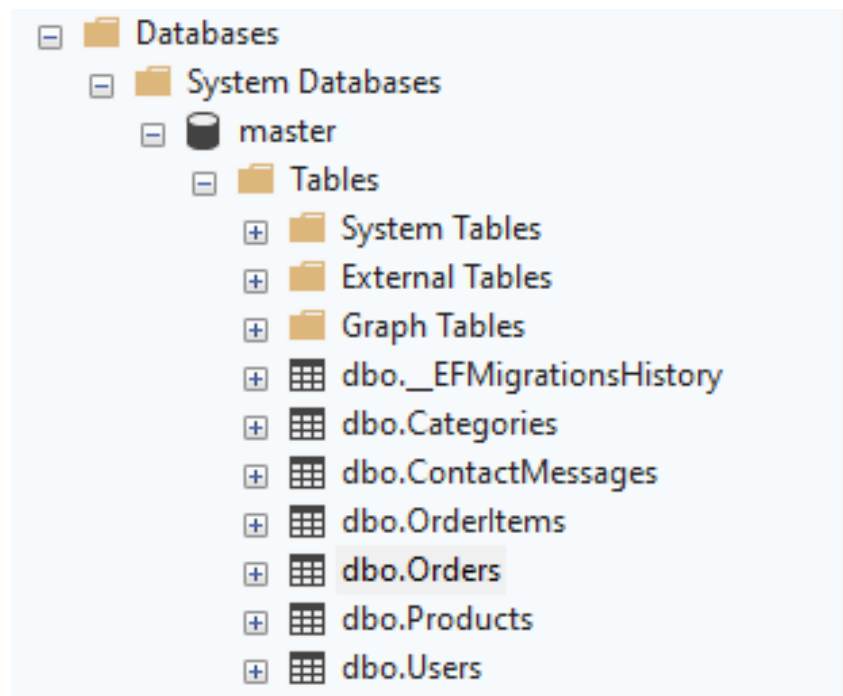
```
{
```

```
var orders = _context.Orders.Include(o => o.Items).ToList();  
return Ok(orders);  
}
```

```
private decimal GetPriceForWeight(Product product, int  
weightGrams)  
{  
    if (weightGrams == 100) return product.Price100g;  
    if (weightGrams == 250) return product.Price250g;  
    if (weightGrams == 500) return product.Price500g;  
    if (weightGrams == 1000) return product.Price1000g;  
    if (weightGrams == 2000) return product.Price2000g;  
    if (weightGrams == 5000) return product.Price5000g;  
    return 0;  
}  
}  
}
```

# Databases Structure

**Tables :**



# Order Details:

SQLQuery1.sql...Admin (b2j) X

1  
2  
3  
4  
5  
6  
7  
8

SELECT TOP (1000) [Id]  
,[CustomerName]  
,[Phone]  
,[Address]  
,[TotalAmount]  
,[OrderDate]  
FROM [master].[dbo].[Orders]

100 % No issues found Ln: 1, Ch: 1 SPC CRLF UTF-

Results Messages

|   | Id | CustomerName | Phone       | Address                                           | TotalAmount | OrderDate                   |
|---|----|--------------|-------------|---------------------------------------------------|-------------|-----------------------------|
| 1 | 1  | Zaib Shaikh  | 09960934181 | house no. 143, rawji nagar, back of power hous... | 110.00      | 2026-09-23 23:55:16.7863898 |
| 2 | 2  | Zaib Shaikh  | 09960934181 | house no. 143, rawji nagar, back of power hous... | 1610.00     | 2026-09-24 07:28:52.4011197 |

# User Login Details:

SQLQuery2.s...Admin (75)) SQLQuery1.sq...Admin (62))

1

SELECT TOP (1000) [Id]

2

, [FullName]

3

, [Email]

4

, [Password]

5

FROM [master].[dbo].[Users]

6

100 % No issues found

Results

Messages

|   | Id | FullName | Email          | Password |
|---|----|----------|----------------|----------|
| 1 | 1  | zaib     | zaib@gmail.com | 123456   |

## Future Scope

The DesiNuts e-commerce platform can be further enhanced by integrating advanced features to elevate the overall shopping and operational experience. Future iterations can incorporate a personalized product recommendation engine powered by machine learning, analyzing user browsing history, purchase patterns, and cart behaviors to suggest relevant dry fruits, nuts, and gift hampers. The system can also introduce subscription-based ordering, allowing customers to schedule automated, recurring deliveries of their favorite items on a weekly or monthly basis.

To streamline business operations and user engagement, the platform can be expanded to include a dedicated mobile application for Android and iOS with push notifications for flash sales, order tracking, and exclusive discounts. Advanced payment gateway integrations, such as multi-currency support, digital wallets, and "buy now, pay later" (BNPL) options, can be added to offer greater flexibility at checkout. Additionally, integrating live order tracking via SMS and WhatsApp updates will keep customers fully informed from dispatch to doorstep delivery.

For long-term growth and administrative efficiency, the platform can incorporate an advanced analytics dashboard providing deep insights into sales trends, inventory turnover, peak shopping hours, and customer retention metrics. Multi-vendor support can also be introduced in the future to allow regional farmers and third-party specialty suppliers to list and sell their organic produce directly through the platform. Finally, deploying the application on a robust cloud infrastructure with a Content Delivery Network (CDN) will ensure high availability, fast asset loading, and seamless scalability to handle traffic spikes during festive seasons.

# BIBLIOGRAPHY

- Mozilla Developer Network (MDN) Web Docs — References and documentation for HTML, CSS, JavaScript, and modern web development standards.
- React Documentation — Official documentation and learning resources for developing user interfaces and managing component state using React.
- Node.js Documentation — Reference material and API guides for building scalable server-side applications using Node.js.
- Express.js Documentation — Documentation for setting up routing, building REST APIs, and managing backend services using Express.js.
- MongoDB Documentation — Reference material for database design, NoSQL schemas, and data management using MongoDB and Mongoose ODM.
- Tailwind CSS Documentation — Guides and reference material used for designing responsive and minimalist user interfaces.
- Postman Documentation — Reference material used for testing, documenting, and validating backend REST APIs.
- GitHub Documentation — Reference material for version control, repository management, and collaborative project development.
- W3Schools — Online tutorials and quick references for HTML, CSS, JavaScript, and other foundational web technologies.
- OpenAI ChatGPT — Used as an AI-assisted learning and development resource for understanding concepts, troubleshooting code, debugging errors, and improving the implementation of the platform.